Chapter 16

Query Optimization

- Why Optimization
- cost-based optimization
- Heuristic optimization（启发式优化）

Why Optimization

- Cost difference between a good and a bad way of evaluating a query can be enormous
  - Equivalent expressions
  - Different algorithms for each operation
- Need to estimate the cost of operations
  - Depends critically on statistical information about relations which the database must maintain
  - Need to estimate statistics for intermediate results to compute cost of complex expressions

Optimization Ways

- cost-based optimization
- Heuristic optimization

cost-based optimization

1. Generating logically equivalent expressions

   Use **equivalence rules** to transform an expression into an equivalent one.

2. Annotating resultant expressions to get alternative query plans
3. Choosing the cheapest plan based on **estimated cost**

Heuristic optimization

- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
  - ★Perform selection early (reduces the number of tuples)
  - ★Perform projection early (reduces the number of attributes)
  - ★Perform most restrictive selection and join operations before other similar operations.
  - ★Some systems use only heuristics, others combine heuristics with partial cost-based optimization.

Equivalence Rules(1)

1.Conjunctive selection operations can be deconstructed into a sequence of individual selections，选择串接律：

$$\delta_{C1\ and\ C2\ldots Cn}（E）=\delta_{C1}（\delta_{C2}\ldots（\delta_{Cn}（E）））\ldots）$$

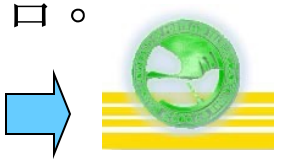
2.Selection operations are commutative，选择交换律

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3.Only the last in a sequence of projection operations is needed, the others can be omitted，投影串接律

$$\pi_{L1}（\pi_{L2}\ldots（\pi_{Ln}（E）））\ldots）=\pi_{L1}（E）$$

$L1 \subseteq L_K$ K为1…n，即L1为其最小的投影属性集合。



Equivalence Rules(2)

4. Selections can be combined with Cartesian products and theta joins.

   a. $\sigma_{\theta}(E_1 \text{ X } E_2) = E_1 \bowtie_{\theta} E_2$

   b. $\sigma_{\theta 1}(E_1 \bowtie_{\theta 2} E_2) = E_1 \bowtie_{\theta 1 \wedge \theta 2} E_2$

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

6.(a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

(b) Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta 1} E_2) \bowtie_{\theta 2 \wedge \theta 3} E_3 = E_1 \bowtie_{\theta 2 \wedge \theta 3} (E_2 \bowtie_{\theta 2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3.

Equivalence Rules(3)

7.The selection operation distributes over the theta join operation under the following two conditions:

(a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

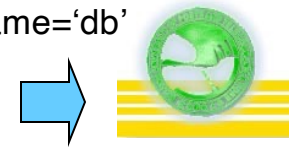
$$\sigma_{\theta 0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 0}(E_1)) \bowtie_{\theta} E_2$$

(b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2.

$$\sigma_{\theta 1 \wedge \theta 2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 1}(E_1)) \bowtie_{\theta} (\sigma_{\theta 2}(E_2))$$

$\delta_{sex='f'}$ （S ⋈SC） = （$\delta_{sex='f'}$ （S） ）⋈ （SC）

$\delta_{sex='f' \cap cname='Db'}$ （S⋈ SC） = （$\delta_{sex='f'}$ （S） ）⋈ ($\delta_{cname='db'}$ （SC） ）



Equivalence Rules(4)

8.The projections operation distributes over the theta join operation as follows:

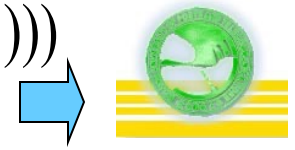
(a) If ϑ involves only attributes from $L_1 \cup L_2$:

$$\prod_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = (\prod_{L_1}(E_1)) \bowtie_\theta (\prod_{L_2}(E_2))$$

(b) Consider a join $E_1 \bowtie_\theta E_2$.

- Let L_1 and L_2 be sets of attributes from E_1 and E_2, respectively .
- Let L_3 be attributes of E_1 that are involved in join condition θ, but are not in $L_1 \cup L_2$, and
- let L_4 be attributes of E_2 that are involved in join condition θ, but are not in $L_1 \cup L_2$.

$$\prod_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = \prod_{L_1 \cup L_2}((\prod_{L_1 \cup L_3}(E_1)) \bowtie_\theta (\prod_{L_2 \cup L_4}(E_2)))$$



$$\prod_{L_1 \cup L_2}(E_1 \overset{\cdot\cdot\cdot}{\bowtie}_{\theta} E_2) = \prod_{L_1 \cup L_2}((\prod_{L_1 \cup L_3}(E_1)) \overset{\cdot\cdot\cdot\cdot\cdot}{\bowtie}_{\theta} (\prod_{L_2 \cup L_4}(E_2)))$$

$$\pi_{sn,\,cn}(S \bowtie SC) = \pi_{sn,\,cn}(\pi_{sn,\,sno}(S) \bowtie \pi_{cn,\,sno}(SC))$$

Equivalence Rules(5)

9. The set operations union and intersection are commutative

$$E_1 \cup E_2 = E_2 \cup E_1$$
$$E_1 \cap E_2 = E_2 \cap E_1$$

10. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$
$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over $\cup$, $\cap$ and $-$.

$$\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - \sigma_\theta(E_2)$$

and similarly for $\cup$ and $\cap$ in place of $-$

Also: $$\sigma_\theta (E_1 - E_2) = \sigma_\theta(E_1) - E_2$$

and similarly for $\cap$ in place of $-$, but not for $\cup$

12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

投影、选择交换律：

（1）π_L（δ_θ（E））$=\delta_\theta$（π_L（E））

条件θ只涉及L中属性；

（2）π_L（δ_θ（E））$=\pi_L$（δ_θ（$\pi_{L,\,Li}$（E））），

Li为θ涉及除去L以外的属性集合。

$\pi_{name,\,sno}$（$\delta_{sno=1}$（S））$=\delta_{sno=1}$（$\pi_{name,\,sno}$（S））

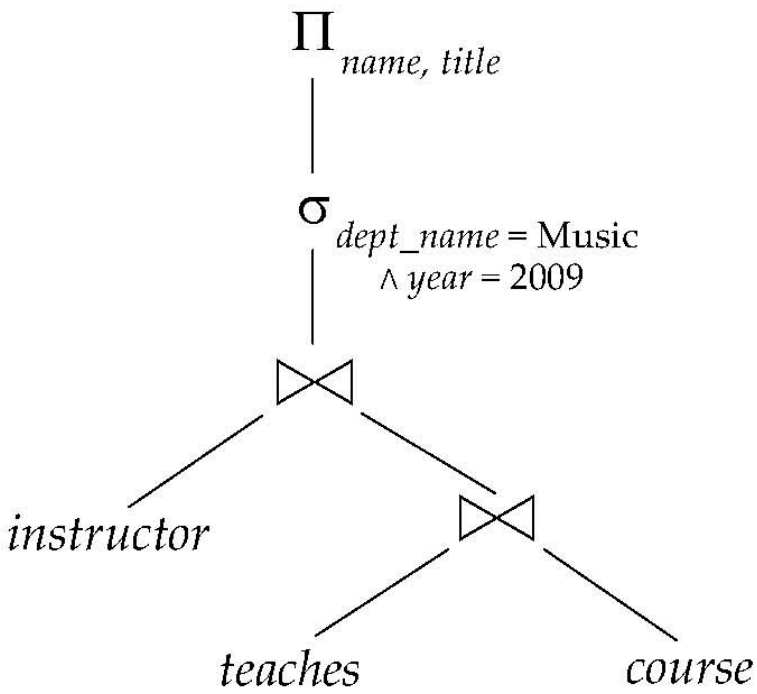
π_{name}（$\delta_{sno=1}$（S））$=\pi_{name}$（$\delta_{sno=1}$（$\pi_{sno,name}$（S））

Transformation Example: Pushing Selections

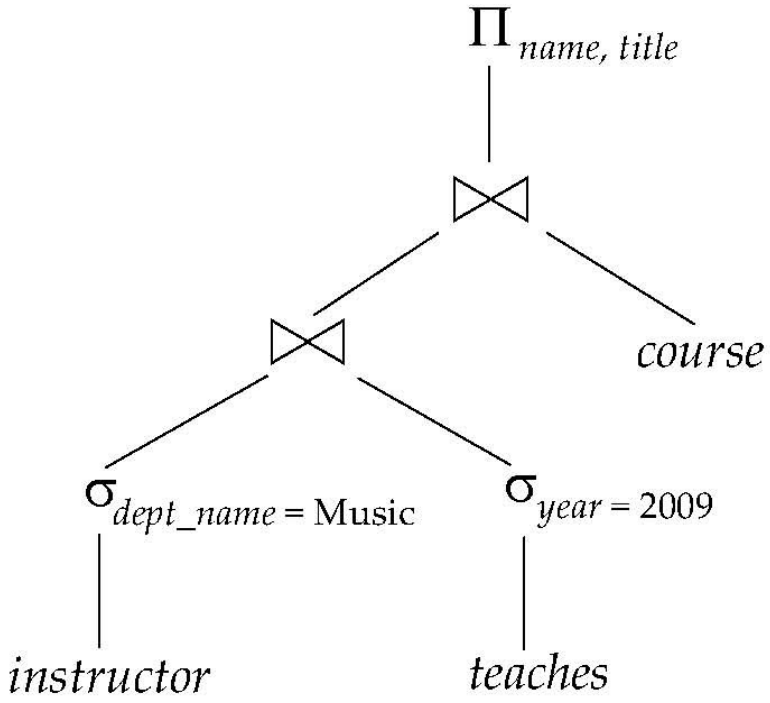
- Query: Find the names of all instructors in the Music department, along with the titles of the courses that they teach
  - $\Pi_{name,\ title}(\sigma_{dept\_name=\text{“Music”}}(instructor \bowtie (teaches \bowtie \Pi_{course\_id,\ title}(course))))$
- Transformation using rule 7a.
  - $\Pi_{name,\ title}((\sigma_{dept\_name=\text{“Music”}}(instructor)) \bowtie (teaches \bowtie \Pi_{course\_id,\ title}(course)))$
- Performing the selection as early as possible reduces the size of the relation to be joined.

- Query: Find the names of all instructors in the Music department who have taught a course in 2009, along with the titles of the courses that they taught
  - $\Pi_{name,\ title}(\sigma_{dept\_name=\ \text{“Music”} \wedge year\ =\ 2009}$
    $(instructor \bowtie (teaches \bowtie \Pi_{course\_id,\ title}\ (course))))$
- Transformation using join associatively (Rule 6a):
  - $\Pi_{name,\ title}(\sigma_{dept\_name=\ \text{“Music”} \wedge gear\ =\ 2009}$
    $((instructor \bowtie teaches) \bowtie \Pi_{course\_id,\ title}\ (course)))$
- Second form provides an opportunity to apply the “perform selections early” rule, resulting in the subexpression

$$\sigma_{dept\_name\ =\ \text{“Music”}}\ (instructor) \bowtie \sigma_{\ year\ =\ 2009}\ (teaches)$$



(a) Initial expression tree



(b) Tree after multiple transformations

Transformation Example: Pushing Projections

- Consider: $\Pi_{name,\ title}(\sigma_{dept\_name=\ \text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course\_id,\ title}(course))$
- When we compute

  $(\sigma_{dept\_name = \text{“Music”}}(instructor) \bowtie teaches)$

  we obtain a relation whose schema is:
  (*ID, name, dept_name, salary, course_id, sec_id, semester, year)*
- Push projections using equivalence rules 8a and 8b; eliminate unneeded attributes from intermediate results to get:

  $\Pi_{name,\ title}(\Pi_{name,\ course\_id}(\sigma_{dept\_name=\ \text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course\_id,\ title}(course))$
- Performing the projection as early as possible reduces the size of the relation to be joined.

Join Ordering Example

- For all relations r_1, r_2, and r_3,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

(Join Associativity)

- If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we compute and store a smaller temporary relation.

- Consider the expression

$$\Pi_{name,\ title}(\sigma_{dept\_name=\text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course\_id,\ title}(course))$$

- Could compute $teaches \bowtie \Pi_{course\_id,\ title}(course)$ first, and join result with

$$\sigma_{dept\_name=\text{“Music”}}(instructor)$$

but the result of the first join is likely to be a large relation.

- Only a small fraction of the university's instructors are likely to be from the Music department
  - it is better to compute

$$\sigma_{dept\_name=\text{“Music”}}(instructor) \bowtie teaches$$

first.

1　选择、投影尽早执行，减少中间结果；

2　笛卡尔乘积和选择组合成连接操作；

3　相同关系的多个选择和投影操作可同时执行，减少扫描；

4　投影和相邻的操作结合，减少为投影而进行单独的I/O操作；

5　存储公共的常用的结果

Steps in Typical Heuristic Optimization

1. Deconstruct conjunctive selections into a sequence of single selection operations (Equiv. rule 1.).
2. Move selection operations down the query tree for the earliest possible execution (Equiv. rules 2, 7a, 7b, 11).
3. Deconstruct and move as far down the tree as possible lists of projection attributes, creating new projections where needed (Equiv. rules 3, 8a, 8b, 12).
4. Execute first those selection and join operations that will produce the smallest relations (Equiv. rule 6).
5. Replace Cartesian product operations that are followed by a selection condition by join operations (Equiv. rule 4a).
6. Identify those subtrees whose operations can be pipelined, and execute them using pipelining.

1　步骤：

（1）　　构造查询树；

（2）　按优化规则、变化规则进行优化。

2　查询树

（1）　　表示关系代数表达式的语法树形结构；

叶节点：关系；

非叶节点：各种关系代数操作；

从底向上执行。

（2） 从SQL构造查询树：

FROM 对应笛卡尔乘积；

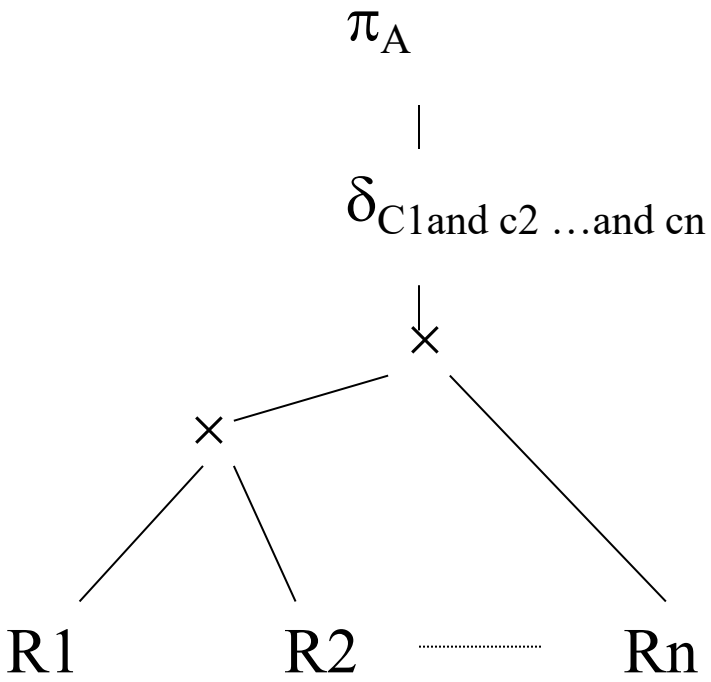
WHERE 对应选择操作；

SELECT 对应投影操作。

Select A

From R1,R2,…,Rn

Where C1 and C2….



3　优化算法

输入：一个关系代数表达式的语法树；

输出：计算表达式的一个优化程序。

执行步骤：对查询树从顶至下优化：

（1） 利用选择串接律，将选择操作分成单个独立；

（2）对于每个选择操作，利用选择的交换律、和投影的交换律、和连接/笛卡尔乘积的分配律、和集合操作的分配律，使其尽可能移到靠近叶节点；

（3）对于每个投影操作，利用投影串接律、和选择的交换律、和连接/笛卡尔乘积的分配律、和集合操作的分配律，使其尽可能移到靠近叶节点或删除某些投影操作；

（4）使用选择、投影的串接律，选择、投影的交换律，将串接的多个选择和投影组合成一个；

（5）利用笛卡尔乘积/连接/结合操作的结合律，按照小关系先执行的原则，安排操作顺序；

（6）组合笛卡尔乘积和相继的选择操作为连接操作；

（7）对每个叶节点加必要的投影操作，消除对查询无用的属性；

（8）分组。每个二元运算（笛卡尔乘积、连接、集合操作）与其直接祖先（不越过别的二元运算节点）的一元运算节点（投影、选择），分为一组，如其子孙节点一直到叶节点皆为选择或投影，也并入该组。使每组的操作可由单个存取程序完成。

Heuristic Optimization Example

Database:

Book (Title, Author, Pubname, Bookno)

Pub (pubname, Pubadrr, Pubcity)

Card (Name, Addr, City, No)

Borrow (No,Bookno, Date)

（1） Create View LBI

As select Title,Author, Book.pubname, Book.Bookno, Name,Addr, City, Card.No, Date

From Book , Borrow , Card

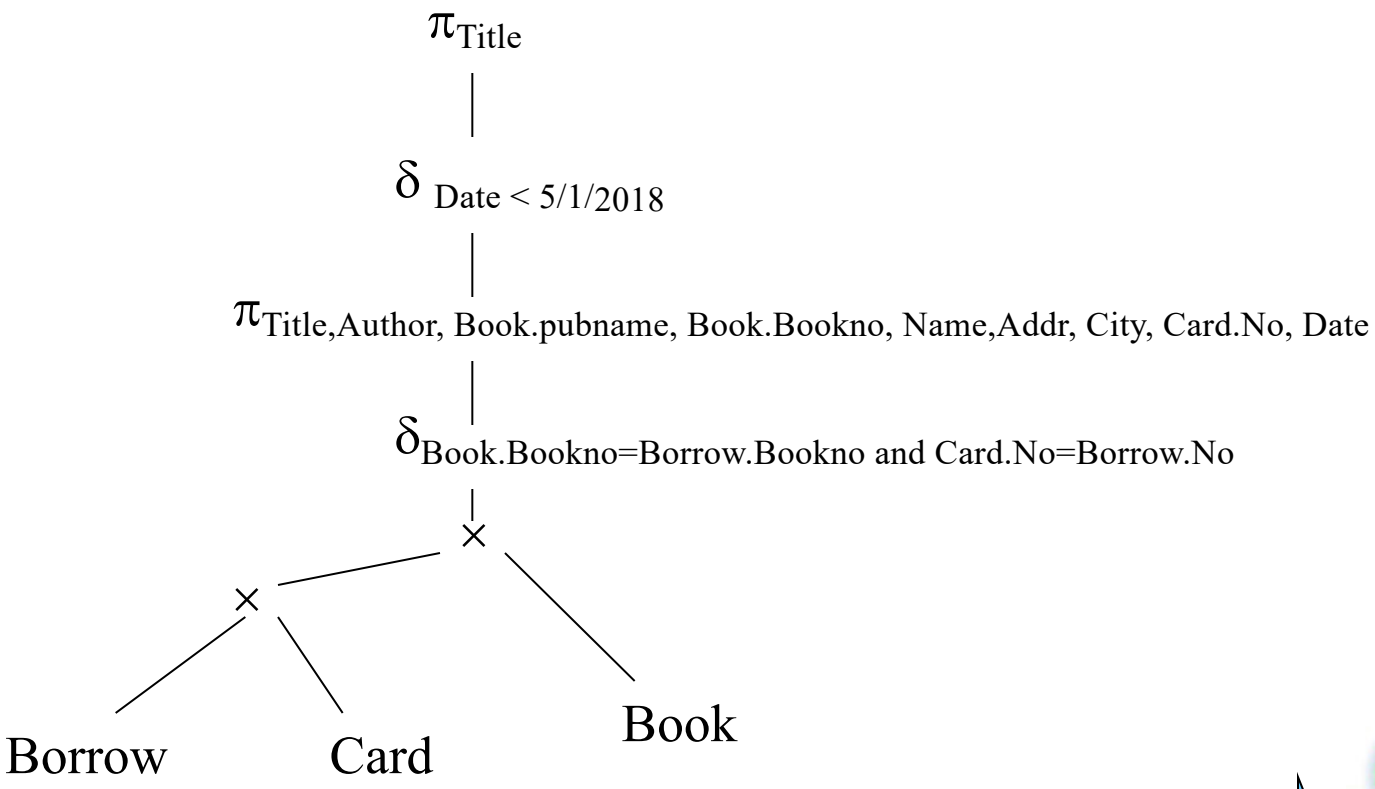
Where Book.Bookno=Borrow.Bookno and Card.No = Borrow.No

（2） Select Title

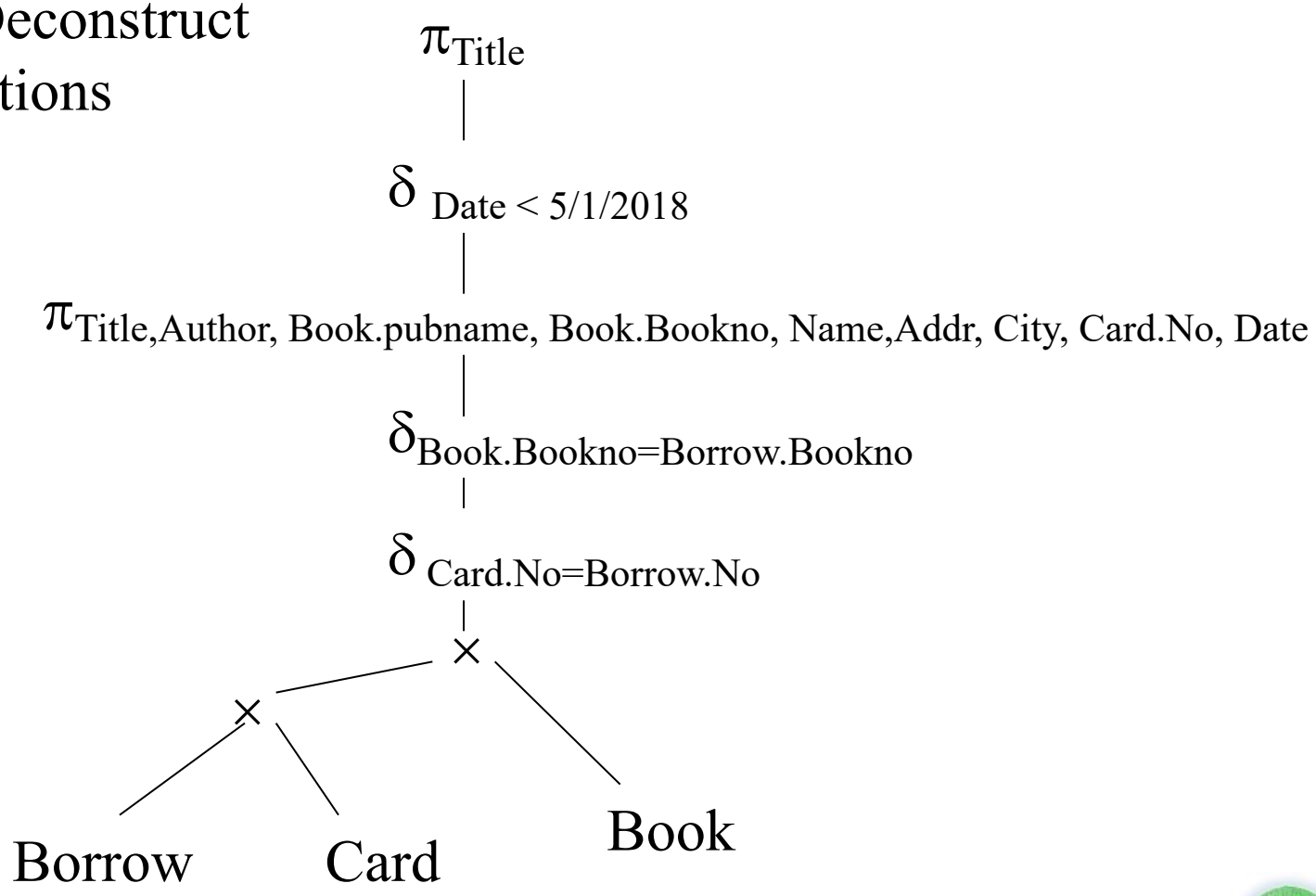
From LBI

Where Date < 5/1/2018

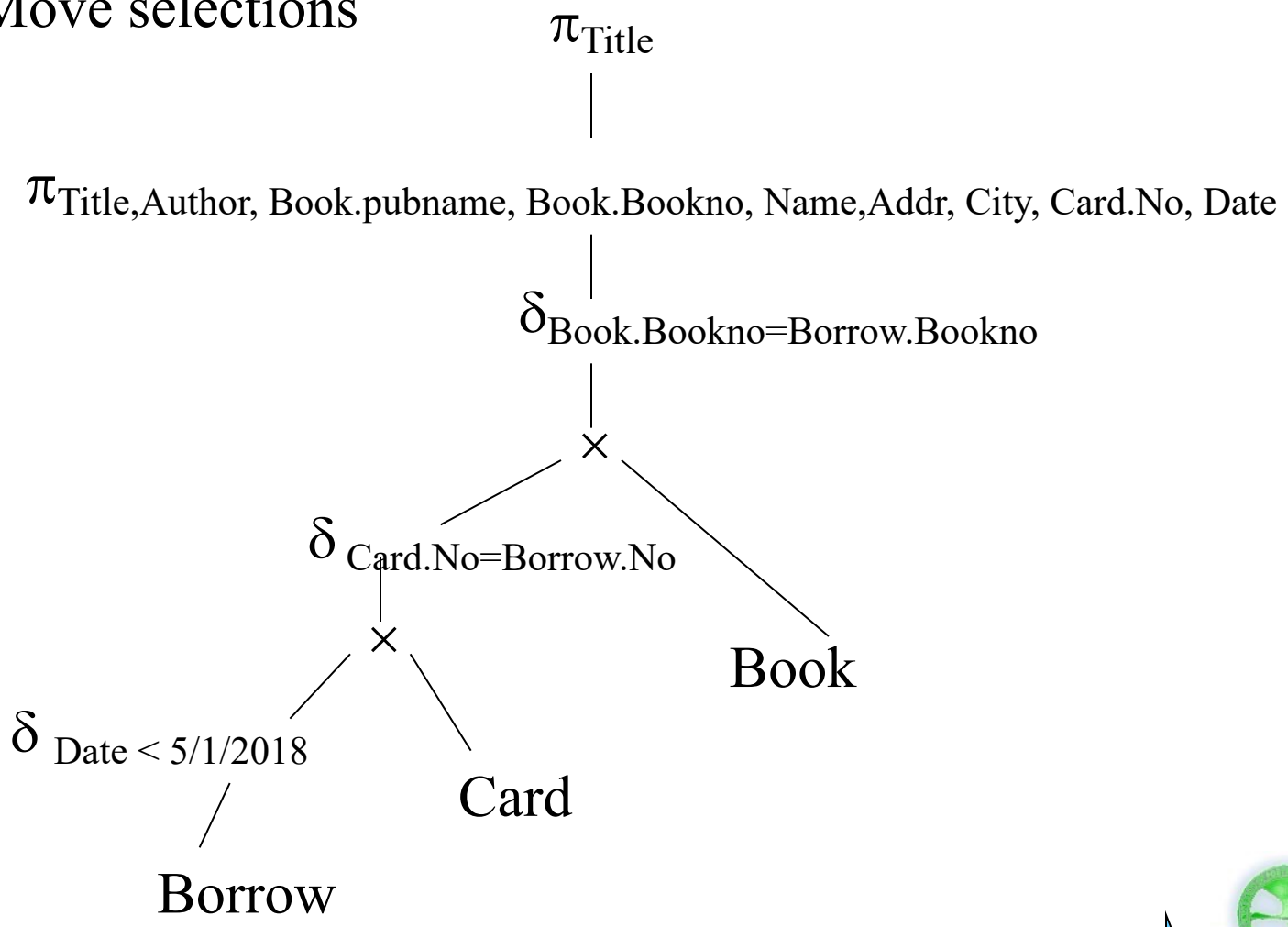
π_{Title}（$\delta_{Date < 5/1/2018}$($\pi_{Title,Author, Book.pubname, Book.Bookno, Name,Addr, City, Card.No, Date}$($\delta_{Book.Bookno=Borrow.Bookno\ and\ Card.No=Borrow.No}$（Book×(Borrow×Card)））)



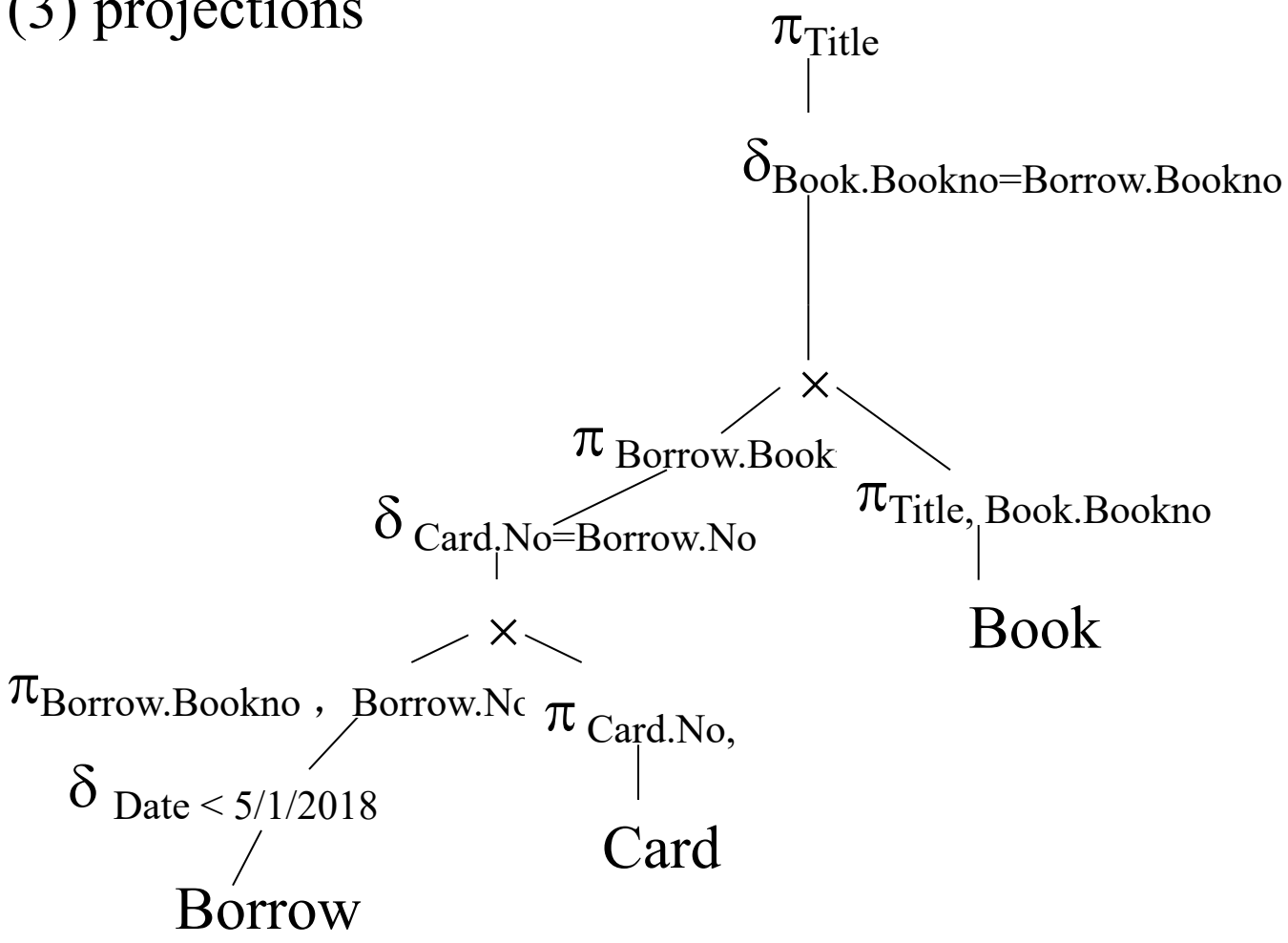
(1) Deconstruct selections

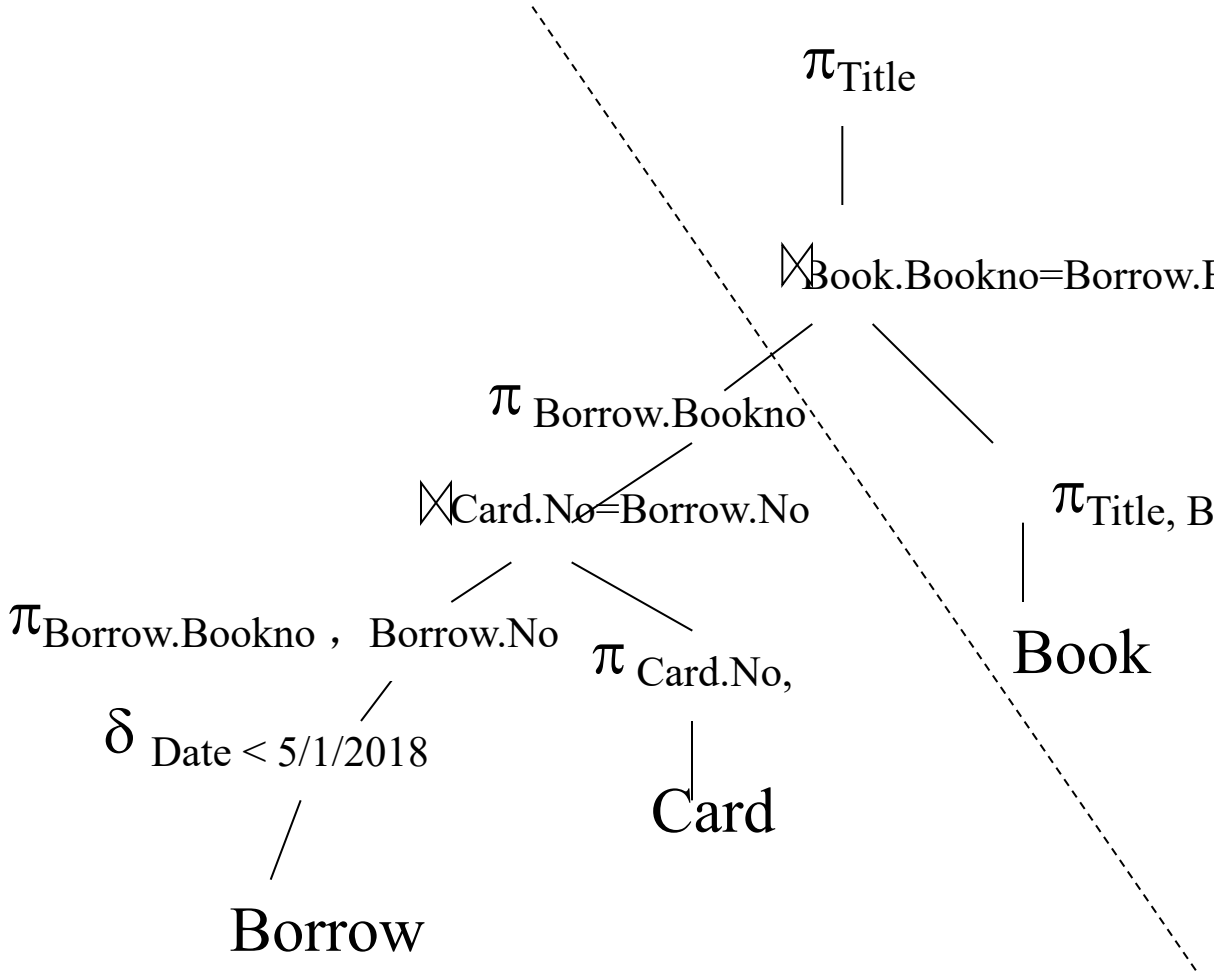


(2) Move selections



(3) projections





$\pi_{Title}(\pi_{Title,\ Book.Bookno}(Book) \bowtie_{Book.Bookno=Borrow.Bookno}(\pi_{Borrow.Bookno}(\pi_{Card.No}(Card) \bowtie_{Card.No=Borrow.No}(\pi_{Borrow.Bookno\ ,\ Borrow.No}(\delta_{Date < 5/1/2018}(Borrow)))))))$

Heuristic Optimization

employee (employee-number, employee-name, sex, age, address, salary, department-number)
project(project-number ,project-name, address, department-number)
works(employee-number, project-number, working-hours)

Find the names of all employees who work for Plane Project and whose working hours is longer than 100 hours .